Inside Concern#append_features

Here is the code in Concern#append_features:

gems/activesupport-4.1.0/lib/active_support/concern.rb

```
module ActiveSupport
  module Concern
    def append_features(base)
      if base.instance_variable_defined?(:@_dependencies)
        base.instance_variable_get(:@_dependencies) << self
        return false
      else
        return false if base < self
        @_dependencies.each { |dep| base.send(:include, dep) }
        super
        base.extend const_get(:ClassMethods) \
          if const_defined?(:ClassMethods)
        # ...
      end
    end

    # ...
```

This is a hard piece of code to wrap your brain around, but its basic idea is simple: never include a concern in another concern. Instead, when concerns try to include each other, just link them in a graph of dependencies. When a concern is finally included by a module that is not itself a concern, roll all of its dependencies into the includer in one fell swoop.

Let's look at the code step by step. To understand it, remember that it is executed as a class method of the concern. In this scope, self is the concern, and base is the module that is including it, which might or might not be a concern itself.

When you enter append_features, you want to check whether your includer is itself a concern. If it has an @_dependencies Class Variable, then you know it is a concern. In this case, instead of adding yourself to your includer's chain of ancestors, you just add yourself to its list of dependencies, and you return false to signal that no inclusion actually happened. For example, this happens if you are ActiveModel::Validations, and you get included by ActiveRecord::Validations.

What happens if your includer is *not* itself a concern—for example, when you are ActiveRecord::Validations, and you get included by ActiveRecord::Base? In this case, you check whether you're already an ancestor of this includer, maybe because you were included via another chain of concerns. (That's the meaning of base < self.) If you are not, you come to the crucial point of the entire exercise: you recursively include your dependencies in your includer. This minimalistic